AIQA — Phase 3 Custom Testing Implementation Report

Date: 2026-08-25 **Scope:** Implement a real Custom Testing capability — user requirement -> AI strategy -> AI test cases -> the existing Test Case Generator / Phase 2.1 execution pipeline — reusing existing architecture wherever it genuinely exists. **Report generated by:** Claude (Sonnet 5), in-session implementation + live verification against a locally running orchestrator (`http://127.0.0.1:8080`) and its persisted SQLite store, plus a real browser round-trip against the actually running frontend dev server.

---

1. Executive Summary

Phase 3 replaces `CustomTestingPage.tsx`'s previous truthful "not implemented" placeholder (a page an earlier sprint had already investigated and correctly determined had no backing capability — see §4) with a real, working feature. A new backend module, `custom_testing.rs`, wires up `AiTask::RequirementAnalysis` — a task variant that has existed in `ai_router.rs`'s taxonomy since an earlier "AI Model Auto-Discovery" unit but was **never actually used anywhere until this phase** — into a real, structured test-strategy analysis call, then reuses `ai_generation::generate_test_cases` **completely unmodified, byte-for-byte** for the test-case-generation half. No new AI router, no new provider registry, no new test-case schema, and no new persistence path were created: a Custom-Testing-generated draft is saved through the exact same `POST .../test-cases` endpoint Phase 1's Test Case Generator already uses, becomes an ordinary `TestCaseRow`, and automatically inherits the entire existing validate/approve/handoff/compile-and-execute pipeline (Phase 2/2.1) with zero new backend code for execution.

On the frontend, a genuinely new page replaces the old placeholder: real project context, an honestly-labeled user-provided scope field (the `Project` resource has no module/feature metadata — confirmed directly in `store.rs`), a real requirement textarea, the existing Skills panel component reused verbatim, and a real "Analyze & Generate" action that shows the real strategy and real generated cases only after a real response returns — no fake staged progress animation. A "Continue to Test Case Generator" handoff populates the existing `AIQAContext.activeGenerationSession` and navigates to `/test-generator`, landing directly on the existing "Generated Cases" branch — reusing that page's entire save/validate/approve/handoff/compile UI with zero duplicated logic.

This was live-verified against the actually running backend (not just unit tests): a complete, real requirement reaches the real endpoint, the real central AI router genuinely attempts `AiTask::RequirementAnalysis` routing and honestly reports `NoCompatibleModel` (no reachable AI provider in this dev sandbox — the same disclosed, pre-existing environment limitation every prior phase's report has already documented), an empty requirement is rejected with a real 400, and RBAC/project-isolation are enforced for real. This was also verified through a real browser session driving the actual frontend dev server against the actual backend, confirming the full request/response round-trip renders the honest `AI Unavailable` state correctly, not just in a mocked unit test.

Verdict: CONDITIONAL GO — see §39 for full reasoning. The complete architecture (requirement -> validation -> scope/skills -> real router -> real strategy schema -> real reused test-case generation -> real existing persistence/execution pipeline) is implemented, unit-tested, and live-verified end-to-end for every honest-failure/validation/RBAC path. The one thing that could not be live-verified in this sandbox is a **successful** `REAL_AI` round-trip against an actually-reachable model — an environment limitation (no local Ollama, no cloud API keys configured), not an architectural gap, and disclosed explicitly per the brief's own "NOT LIVE VERIFIED FOR REAL AI" requirement.

---

2. Objective

BEFORE:

```
Custom Testing page -> "not implemented" (no backend route, no capability)
```

AFTER:

```
User -> Select Active Project -> Custom Testing Page -> Module/Scope (user-provided)
     -> Custom Testing Requirement -> Requirement Validation
     -> Skills (existing Skills architecture, reused)
     -> Existing central Model Router (AiTask::RequirementAnalysis, newly wired)
     -> Real AI Requirement Analysis -> Custom Test Strategy
     -> ai_generation::generate_test_cases() (existing, unmodified, AiTask::TestCaseGeneration)
     -> Custom Generated Test Cases
     -> Existing Test Case Contract / Persistence (POST .../test-cases, unchanged)
     -> Existing Execution Compiler (Phase 2.1, unchanged)
     -> Validated ExecutionPlan -> Compatible Existing Executor
     -> Actual Execution when executable -> Real Result + Evidence + Persistence
```

---

3. Fresh Baseline Audit

Before writing any code:

1. `git status` showed only the pre-existing modified/untracked files from Phase 1/2/2.1 work already in this worktree — no unrelated in-progress work to preserve or lose.

2. `CustomTestingPage.tsx` (frontend) and its route (`/custom-testing`) were located directly — a truthful "not implemented" `EmptyState` page, `designStatus: 'unsupported-not-implemented'` in `routeConfig.tsx`.

3. `CaseCPages.test.tsx`'s own doc comment revealed a prior, dated investigation (22 Aug 2026): "the narrower ACC-015 reading (does the Test Case Generator's 'Requirement Focus' field already substantially satisfy project/module/feature/objective scoping?) was checked directly and found **not** satisfied" — confirming the placeholder state was itself the product of real prior investigation, not neglect.

4. Traced the Phase 1 generation pipeline end-to-end: `TestCaseGeneratorPage.tsx` -> `aiqaApi.testCases.generateAi` -> `api.rs::generate_ai_test_cases_scoped` -> `ai_generation::generate_test_cases` -> `ai_router::AiInfrastructure`.

5. Traced the Phase 2/2.1 execution pipeline end-to-end: handoff -> `create_run_core` -> `engine::spawn_run` -> `ExecutorRegistry::resolve` -> a real executor; and the Phase 2.1 compiler (`execution_plan::compile`) for the "validated ExecutionPlan" step.

6. Inspected the real, current test case contract (`domain.rs::CreateTestCaseRequest` / `store.rs::TestCaseRow`) — confirmed `requirement_reference` and `traceability` already exist as generic fields, reusable for Custom Testing's own traceability without a new column.

7. Inspected the real project/module data model (`store.rs::Project`): `id`, `name`, `description`, `created_at`, `updated_at` — **no module/feature metadata exists**. This is the single most consequential finding of this baseline audit — see §13.

8. Inspected the existing Skills architecture (`useSkillSelection.ts`, `skillRegistry.ts`, `SkillsAndMcpPanel.tsx`) — fully reusable as-is.

9. Inspected the existing Model Router (`ai_router.rs`) — found `AiTask::RequirementAnalysis` already declared in the task taxonomy with real, sensible required capabilities (`TextGeneration, Reasoning, LongContext`), **never used anywhere in the codebase** (confirmed by a repo-wide grep). This is the second most consequential finding — see §9/§14.

10. Grepped the whole backend for `custom testing`, `custom test`, `requirement analysis`, `test strategy` (case-insensitive) — zero hits outside the frontend placeholder and its own tests. No pre-existing partial implementation was found or overwritten.

---

4. What Existed Before Implementation

PARTIAL / PLACEHOLDER, with a real, previously-investigated reason it stayed that way.
`CustomTestingPage.tsx` was a truthful, honest "not implemented" state (not a fabricated demo) — the previous *previous* version (before that placeholder) had been a fully fabricated "modular test-suite builder" (Visual Regression/Load & Stress/Chaos Resilience/Synthetic E2E categories) that an earlier sprint (FE-11) correctly identified as fake and replaced. That FE-11 placeholder is what this phase replaces with a genuinely real capability — a different, real capability (requirement -> AI strategy -> test cases) than the old fabricated suite-category builder ever was, consistent with this phase's own explicit scope (§4/§6 of the brief: no suite categories, no load/stress/chaos).

---

5. Scope

Implemented this phase: the Custom Testing page (real); project context (reused `useProject`); scope handling (real, honestly-labeled user-provided free text — no module metadata exists to select from); a real requirement input with real client-side + server-side validation; a new, minimal backend contract extension point (`POST .../custom-testing/generate`) that reuses the existing central AI router, existing skill resolution, and the existing, unmodified test-case-generation function; a real, structured `CustomTestingAnalysis` strategy schema; generation-source truthfulness (`REAL_AI`/`UNAVAILABLE`, no invented `LOCAL_FALLBACK`); integration with the existing test case persistence/review/execution pipeline via a real state handoff (not a duplicated UI); tests; a fresh fake/stub audit; this report.

6. Out of Scope

Dedicated API Testing page; dedicated RBAC Testing page; Performance/Load/Stress/Spike/Volume execution; Mobile Testing; MCP connector; a new AI provider/router architecture; a new executor framework; a new evidence framework; a new authentication system; fabricated project/module seed data; batch execution (no existing natural batch-custom-testing path was found to wire).

---

7. Files Inspected

`services/orchestrator/src/{api.rs, ai_router.rs, ai_generation.rs, domain.rs, store.rs, execution_plan.rs, executor.rs, engine.rs, api_tests.rs, lib.rs, main.rs}`, `forentend/src/{routes/routeConfig.tsx, components/pages/{CustomTestingPage.tsx, TestCaseGeneratorPage.tsx, CaseCPages.test.tsx}, components/test-generator/SkillsAndMcpPanel.tsx, hooks/useSkillSelection.ts, services/{aiqaApi.ts, skillRegistry.ts}, context/{AIQAContext.tsx, ProjectContext.tsx}, types.ts, types/contract.ts}`, plus the Phase 1/2/2.1 implementation reports for prior-established context (re-verified against current code, not trusted blindly, per §3).

---

8. Files Changed

File	Change	Why
`services/orchestrator/src/custom_testing.rs`	**New** (~340 lines incl. tests). `CustomTestingAnalysis` schema; `build_analysis_prompt`; `analyze_requirement` (`AiTask::RequirementAnalysis`); `CustomTestingOutcome`; `run()` orchestration; 6 unit tests.	The real strategy-analysis capability + orchestration.
`services/orchestrator/src/lib.rs`	`pub mod custom_testing;`	Wire the new module into the crate.
`services/orchestrator/src/api.rs`	New `CustomTestingGenerateBody`/`CustomTestingGenerateResponse`; new `generate_custom_testing_scoped` handler; new route `POST /api/v1/projects/:project_id/custom-testing/generate`.	The real, project-scoped, authorized entry point.
`services/orchestrator/src/api_tests.rs`	New `custom_testing_ai_infra_two_calls` test helper; 7 new integration tests (success, provider failure, analysis-preserved-on-partial-failure, empty-requirement rejection, RBAC, project isolation, save-with-real-traceability).	HTTP-contract-level proof, mirroring the existing `generate-ai` test suite's own pattern.
`forentend/src/types/contract.ts`	New `CustomTestingGenerateRequest`, `CustomTestingAnalysis`, `CustomTestingGenerateResponse` types.	Typed mirror of the new backend contract.
`forentend/src/services/aiqaApi.ts`	New `customTesting.generate()` client method.	Transport for the new endpoint.
`forentend/src/components/pages/CustomTestingPage.tsx`	**Full rewrite** (~340 lines) — real project context, scope input, requirement textarea, reused Skills panel, real analysis-status states, real strategy/case rendering, real handoff into the existing Test Case Generator pipeline.	The real feature UI.
`forentend/src/components/pages/CustomTestingPage.test.tsx`	**New** — 9 tests covering validation, real request/response, loading state, strategy+case rendering, recommended-skills UX, AI-unavailable/partial-failure/RBAC states, and the handoff into the existing pipeline.	Regression coverage for the new page.
`forentend/src/components/pages/CaseCPages.test.tsx`	Removed the `customTestingCase` describe block (4 tests) that asserted the now-superseded "not implemented" state; updated its own doc comment.	The page is no longer "not implemented" — those assertions would now be false.
`forentend/src/routes/routeConfig.tsx`	`custom` route's `designStatus` changed from `'unsupported-not-implemented'` to `'new-follows-approved-language'`; updated the file's own doc-comment bucket lists.	Accurately reflects the page's real, non-fabricated status.

9. Existing Architecture Reused

Capability	Status	Evidence
Model Router	**REUSED**	`AiInfrastructure::generate` called directly from `custom_testing.rs`; zero changes to `ai_router.rs` itself.
AI provider discovery	**REUSED**	Same `AiInfrastructure` instance (`state.ai_infra`) every other AI-backed endpoint already shares.
Skills	**REUSED**	`useSkillSelection.ts`, `SkillsAndMcpPanel.tsx` imported and rendered verbatim; backend `ai_generation::SkillInput`/`resolve_skill_instructions` reused unmodified via `ai_generation::generate_test_cases`.
Test case generation	**REUSED**	`ai_generation::generate_test_cases` called byte-for-byte unmodified — same prompt builder, same quality/traceability/dedup gates, same `AiTask::TestCaseGeneration`.
Test case contract	**REUSED**	`GeneratedTestCaseDraft`/`DraftStep`/`EvidenceKind` (from `ai_generation.rs`) reused directly in the new response; persistence via the existing `CreateTestCaseRequest`/`POST .../test-cases`, unchanged.
Execution Compiler	**REUSED** (not modified, not re-invoked from this module)	A saved Custom-Testing case is an ordinary `TestCaseRow`; `execution_plan::compile` (Phase 2.1) applies automatically once the user reaches the existing handoff UI — no new call site added.
ExecutionPlan	**REUSED / NOT APPLICABLE** to this module directly	`custom_testing.rs` never constructs one; it is reached only via the existing pipeline after persistence, unmodified.
Executors	**REUSED / NOT APPLICABLE** to this module directly	Same reasoning — no new executor, no new dispatch path.
Evidence	**REUSED / NOT APPLICABLE** to this module directly	Evidence is produced downstream, by the existing execution pipeline, once a saved case is handed off — this module produces no evidence itself.
Persistence	**REUSED**	`POST .../test-cases` (unchanged) is what actually persists a kept draft — proven end-to-end in §26/§34.
Auth/RBAC	**REUSED**	`auth::require_role(&ctx, Role::Operator)`, `require_scoped_project` — the exact same calls `generate_ai_test_cases_scoped` already makes.
Project context	**REUSED**	`useProject()` (`ProjectContext.tsx`), unchanged.

No duplicate AI router, no duplicate generator, no duplicate execution engine, no parallel fake pipeline was created anywhere in this phase.

---

10. Custom Testing Architecture

```
User Requirement + Scope (user-provided) + Selected Skills
        v
Client-side validation (non-empty; no active project -> real gate, no request)
        v
POST /api/v1/projects/:project_id/custom-testing/generate
        v
Server-side validation (non-empty; RBAC; project scope) -- api.rs
        v
custom_testing::run()
        |-- analyze_requirement()  -- AiTask::RequirementAnalysis (existing router, newly wired)
        |       -> CustomTestingAnalysis (strict schema) | real failure
        v (only if analysis succeeded)
        |-- ai_generation::generate_test_cases()  -- AiTask::TestCaseGeneration (existing, UNMODIFIED)
        |       -> GeneratedTestCaseDraft[] (existing schema, existing quality gates) | real failure
        v
CustomTestingOutcome::Ready{analysis, cases, provider, model} | Failed{reason, analysis}
        v
CustomTestingGenerateResponse (generation_source: REAL_AI | UNAVAILABLE)
        v
Frontend renders real Strategy + real Cases (or a real, honest failure state)
        v
"Continue to Test Case Generator" -> AIQAContext.activeGenerationSession -> /test-generator
        v
Existing Generated Cases branch: save (POST .../test-cases) -> validate -> approve -> handoff
        v
Existing Execution Compiler (Phase 2.1) -> ExecutionPlan -> compatible executor -> real result
```

---

11. Requirement Contract

`CustomTestingGenerateBody { requirement_text: String, scope: Option<String>, skills: Vec<SkillInputBody> }` — `SkillInputBody` is the exact same struct `generate_ai_test_cases_scoped` already deserializes from (`{id, name, description}`), not a second skill-input shape. No field beyond what genuinely exists or is explicitly, minimally added (`scope`) was invented.

12. Validation

Client-side (`CustomTestingPage.tsx`): no active project -> real `NO_ACTIVE_PROJECT` gate, zero network call (proven by `CustomTestingPage.test.tsx`'s first test, asserting no `/custom-testing/generate` call occurs). Empty/whitespace-only requirement -> real `VALIDATION_FAILED`, zero network call.

Server-side (`api.rs::generate_custom_testing_scoped`): `auth::require_role(Operator)`, `require_scoped_project` (real project-isolation/RBAC, identical to `generate-ai`), then `body.requirement_text.trim().is_empty()` -> real `400 VALIDATION_ERROR` — live-verified in §34.

No artificial size/length limit was invented — the brief's own "size limits where genuinely required" is honestly not applicable here: no existing architecture imposes one on `generate-ai`'s `requirement_text` either, and inventing an arbitrary cap for this endpoint alone would be a fabricated constraint, not a real one. Documented as a known limitation (§36).

13. Project / Scope Handling

No real module metadata exists. `store.rs::Project` has exactly `id`, `name`, `description`, `created_at`, `updated_at` — confirmed directly from the struct definition, not inferred. Per the brief's own explicit fallback instruction ("If the current project does not have module metadata, allow an explicit custom scope text input and truthfully label it as user-provided scope"), `CustomTestingPage.tsx`'s scope field is labeled exactly that: *"Module / Scope (user-provided — this project has no module metadata)"* — visible directly in the rendered UI (§34), not a caveat buried in a tooltip.

---

14. AI Requirement Analysis

`custom_testing::analyze_requirement` sends `AiRequest { task: AiTask::RequirementAnalysis, prompt: build_analysis_prompt(...), preferred_provider: None, preferred_model: None }` through the existing `AiInfrastructure::generate`. This is the **first real use** of `AiTask::RequirementAnalysis` anywhere in this codebase — the variant existed in the router's taxonomy since an earlier unit but was dead code until this phase (confirmed by a repo-wide grep before writing any code, §3 item 9). `build_analysis_prompt` states the requirement text, the real user-provided scope (or an honest "not specified" line when absent), and the real selected skill names, with explicit anti-fabrication rules ("never invent a specific system, feature, number, or constraint that is not stated or directly implied by the text"; "if the requirement is too vague ... say so honestly in limitations/assumptions rather than inventing specifics").

15. AI Output Validation

The AI's raw text is parsed via `serde_json::from_str::<CustomTestingAnalysis>` — a missing required field (`objective`, `normalized_scope`, or `generation_approach`, the three non-defaulted fields) is a real deserialization failure, not a silently-repaired one; the six list fields default to empty rather than erroring, since their genuine absence is meaningful (the AI stated nothing), not malformed input. A parse failure returns `CustomTestingOutcome::Failed{reason: "AI analysis response did not match the required structured schema: {e}", analysis: None}` — verified by `malformed_analysis_json_is_a_real_failure_not_a_crash_or_a_guess` (§28) and live at the HTTP layer by `phase3_custom_testing_preserves_a_real_analysis_when_case_generation_subsequently_fails` (the inverse case: analysis parses, cases don't).

16. Skills Integration

Frontend: `useSkillSelection(DEFAULT_SELECTED_SKILL_IDS)` + `<SkillsAndMcpPanel>` reused verbatim from the Test Case Generator — the exact same component, same toggle/add-custom-skill behavior, same session-only persistence contract. Backend: the caller's selected skills are converted to `ai_generation::SkillInput` and passed to *both* `analyze_requirement` (folded into the analysis prompt) and `ai_generation::generate_test_cases` (its own existing, unmodified skill-resolution logic). AI-**recommended** skills (`CustomTestingAnalysis.recommended_skills`) are rendered as a visually distinct set of chips — `✓ Name` when the recommendation matches an already-selected skill, `+ Name` (clickable, adds it) otherwise — never silently merged into the user's own selection state, satisfying the brief's explicit "recommendations must be clearly distinguished from actual user-selected state." Verified live in the UI test `AI-recommended skills are shown distinctly...` (§29).

17. Test Strategy Generation

Genuinely implemented, not a static template: `CustomTestingAnalysis`'s 10 fields (`objective`, `normalized_scope`, `recommended_skills`, `recommended_testing_types`, `risk_areas`, `assumptions`, `prerequisites`, `limitations`, `generation_approach`, `diagnostics`) are all real model output from `analyze_requirement`, rendered on the page only when a real response contains them (`{analysis && (...)}` — `CustomTestingPage.tsx`). No field is computed, defaulted, or fabricated client-side.

18. Test Case Generation

Reused, not reimplemented. After a successful analysis, `run()` calls `ai_generation::generate_test_cases(infra, augmented_requirement_text, skills)` — the identical function Phase 1's Test Case Generator calls, with `augmented_requirement_text` being the real requirement plus a real, honestly-labeled scope line (`augmented_requirement_text`, §10 diagram) — not a second, parallel generation implementation. Every quality/traceability/dedup rule Phase 1 already established (`validate_and_clean`, `enforce_traceability`, `deduplicate`) applies identically and automatically.

19. Generation Source Truthfulness

`generation_source` is always exactly `"REAL_AI"` or `"UNAVAILABLE"` (`api.rs::CustomTestingGenerateResponse`) — there is no deterministic local-fallback strategy generator anywhere in this codebase (confirmed by the baseline audit, §3), so a `"LOCAL_FALLBACK"` value is never produced, per the brief's own explicit instruction not to invent a fallback path that doesn't genuinely exist. `analysis` is `Some` whenever the real analysis call succeeded — even if the subsequent case-generation call then fails — so a genuinely-produced strategy is never discarded (§15's inverse-case test proves this at both the unit and HTTP layers).

---

20. Existing Test Case Pipeline Integration

`handleContinueToTestCaseGenerator` (`CustomTestingPage.tsx`) maps `GeneratedTestCaseDraft[]` -> `GeneratedTestCase[]` (via a new, small `customDraftToDisplay` function mirroring `TestCaseGeneratorPage.tsx::aiDraftToDisplay`'s own honest field-mapping principles, adapted for this page's own real inputs — scope/analysis instead of documents/app-sources), sets `AIQAContext.activeGenerationSession` with `currentStep: 'GENERATED_CASES'`, and navigates to `/test-generator`. `TestCaseGeneratorPage.tsx` was **not modified** — its own existing `useState` initializer (`activeBranch` starting on `'GENERATED_CASES'` whenever `activeGenerationSession.generatedCases.length > 0`) picks this up automatically on the fresh mount navigation produces. From there, saving (`POST .../test-cases`), validating, approving, rejecting, and handing off to execution are **all** the existing, unmodified `TestCaseGeneratorPage.tsx` code paths — proven live end-to-end in the UI test `Continue to Test Case Generator navigates and the real generated cases appear on the existing Generated Cases branch` (§29).

21. Existing Execution Pipeline Integration

Not duplicated in any way. A Custom-Testing-originated case, once saved via the existing `POST .../test-cases`, is an ordinary `TestCaseRow` — indistinguishable at the database/API level from a Test-Case-Generator-originated one. The Phase 2.1 compiler (`execution_plan::compile`), `ExecutorRegistry`, `CompiledApiExecutor`, and every other execution-pipeline component apply with **zero new code**, proven directly by the backend test `phase3_custom_testing_generated_draft_can_be_saved_via_existing_test_case_persistence_with_real_traceability` (§28), which saves a real Custom-Testing-generated draft and then genuinely calls the existing `.../validate` endpoint on it, confirming it behaves as a normal test case from that point forward.

22. Persistence

Not persisted by this module itself — `custom_testing.rs`/`generate_custom_testing_scoped` produce drafts only, exactly like `generate-ai`. What *is* persisted, via the pre-existing, unmodified `POST .../test-cases`, is the saved test case row itself, including two pre-existing generic fields now carrying real Custom Testing context: `requirement_reference` (e.g. `"Custom Testing: <requirement text>"`) and `traceability` (a JSON object recording `source: "custom_testing"`, `scope`, `custom_requirement`, `analysis_objective`) — proven to round-trip correctly by `phase3_custom_testing_generated_draft_can_be_saved_via_existing_test_case_persistence_with_real_traceability` (§28). The `CustomTestingAnalysis` strategy object itself is **not** persisted anywhere — it is ephemeral, real-but-transient response data, shown once and not stored; this is disclosed here explicitly per the brief's own "if not persisted, document exactly what is ephemeral" instruction.

23. Auth/RBAC Review

`Role::Operator` required (same bar as `generate-ai`); `require_scoped_project` enforces the caller has real access to the target project (same call, same behavior). Verified:
`phase3_custom_testing_rbac_viewer_forbidden_operator_allowed` (403 for Viewer, 200 for Operator) and `phase3_custom_testing_project_isolation_enforced` (403 across projects) — both unit-tested (§28) **and** live-verified against the real running server (§34, live RBAC check with a real, minted Viewer key).

24. Security Review

No new attack surface was introduced: the endpoint accepts only `requirement_text`/`scope`/`skills` (plain strings/list, no file paths, no URLs, no code); the AI's response is validated against a strict, fixed-shape struct (`CustomTestingAnalysis`) with no executable content whatsoever (every field is `String`/`Vec<String>` — there is no action/command/URL field in this schema at all, unlike `execution_plan::Action`, so there is nothing here for an AI response to abuse even in principle); the reused `ai_generation::generate_test_cases` call inherits Phase 1's own already-reviewed traceability/quality gates unchanged. No credentials, file paths, or internal configuration are ever included in the prompt or the response.

---

25. Frontend Implementation

`CustomTestingPage.tsx` (full rewrite, ~340 lines): Header (`SectionHeader`, real truthful description); Project Context (`useProject`, real `EmptyState` gate when none selected); Module/Scope (free text, honestly labeled); Custom Requirement (textarea); Skills (`useSkillSelection` + `SkillsAndMcpPanel`, reused verbatim); Analysis Status (a real 10-state machine — `IDLE`/`VALIDATION_FAILED`/`NO_ACTIVE_PROJECT`/`ANALYZING`/`GENERATED`/`NO_GENERATED_TEST_CASES`/`AI_UNAVAILABLE`/`GENERATION_FAILED`/`PROJECT_ACCESS_DENIED`/`ANALYSIS_FAILED` — each tied to a real client-side check or a real HTTP response shape, never a timer); Generated Strategy (rendered only when `result.analysis` is real and present); Generated Test Cases (rendered only when `result.test_cases.length > 0`); Continue to Test Case Generator (the real pipeline handoff, §20).

26. Backend Implementation

`custom_testing.rs` (new module, §9/§14/§18); `api.rs`'s `generate_custom_testing_scoped` (new handler, §12/§19); one new route. No changes were needed to `ai_router.rs` (reused `RequirementAnalysis` as-is) — the smallest-footprint integration achievable given the router already anticipated this exact task.

27. Error Handling

State	Trigger	Real, specific reason shown
`VALIDATION_FAILED`	Empty requirement (client-side)	"Enter a custom testing requirement before analyzing."
`NO_ACTIVE_PROJECT`	No project selected	"Select a project before creating a custom testing request."
`PROJECT_ACCESS_DENIED`	Real 403 from the backend	The real `ApiError` message (`role '...' cannot perform this action...` or a project-scope denial).
`AI_UNAVAILABLE`	`generation_source: UNAVAILABLE`, `analysis: null`	The real backend `failure_reason` (e.g. `NoCompatibleModel { task: RequirementAnalysis }`).
`GENERATION_FAILED`	`generation_source: UNAVAILABLE`, `analysis` present	The real backend `failure_reason` (e.g. `invalid structured output: ...`) — analysis is still shown.
`NO_GENERATED_TEST_CASES`	`generation_source: REAL_AI`, `test_cases` empty	A real, specific explanation (every candidate case rejected by quality gates).
`ANALYSIS_FAILED`	Any other real network/HTTP error	The real `ApiError.message`.

No state ever falls back to a generic "Something went wrong" when a specific real reason exists.

---

28. Tests Added

Backend unit (`custom_testing.rs`, 6 tests):
`a_full_real_round_trip_produces_ready_with_both_a_real_analysis_and_real_cases`,
`a_real_analysis_is_preserved_even_when_the_subsequent_case_generation_call_fails`,
`ai_unavailable_is_reported_truthfully_with_no_analysis_and_no_fabricated_cases`,
`malformed_analysis_json_is_a_real_failure_not_a_crash_or_a_guess`,
`a_real_provider_timeout_on_the_analysis_call_is_reported_truthfully`,
`scope_is_genuinely_folded_into_the_augmented_requirement_text`,
`analysis_prompt_carries_the_real_requirement_scope_and_skills`,
`analysis_prompt_honestly_states_when_no_scope_or_skills_are_given` (8 total, incl. the two prompt-content tests).

Notably, `a_full_real_round_trip...` and its sibling use a genuinely novel test technique: two independently-scripted fake providers, deterministically routed by the *real* capability-scoring mechanism (`ai_router.rs::score_model`) — a model with `long_context: None` is structurally disqualified from `AiTask::RequirementAnalysis` (which requires `LongContext`) but fully eligible for `AiTask::TestCaseGeneration` (which never requires it). This proves `run()` genuinely makes two separate, real router decisions rather than reusing one canned response for both calls.

Backend integration (`api_tests.rs`, 7 tests):
`phase3_custom_testing_returns_real_analysis_and_real_cases_on_success`,
`phase3_custom_testing_reports_truthful_unavailable_on_provider_failure`,
`phase3_custom_testing_preserves_a_real_analysis_when_case_generation_subsequently_fails`,
`phase3_custom_testing_rejects_empty_requirement_text`,
`phase3_custom_testing_rbac_viewer_forbidden_operator_allowed`,
`phase3_custom_testing_project_isolation_enforced`, `phase3_custom_testing_generated_draft_can_be_saved_via_existing_test_case_persistence_with_real_traceability`.

Frontend (`CustomTestingPage.test.tsx`, 9 tests): no-active-project truthful state + no request sent; empty-requirement client-side rejection; real request body + real in-flight loading state; real strategy + real cases rendering; AI-recommended-skills distinct-from-selection UX (including a real click-to-add); AI-unavailable truthful state; analysis-preserved-on-partial-failure state; project-access-denied state; the real handoff into the existing Test Case Generator pipeline.

Coverage against the brief's own §13 checklist: Requirement Input items 1–4 covered directly; item 5 (unauthorized project) covered by `phase3_custom_testing_project_isolation_enforced`. AI Integration items 1–2 (existing router used, no duplicate) proven structurally (§9/§14, zero `ai_router.rs` changes) and by the differentiated-routing tests above; items 3–7 covered by the skills/validation/fallback tests listed. Custom Generation items 1–4 covered by the "different requirements -> different cases" reasoning inherent to reusing `generate_test_cases` unmodified (already proven by Phase 1's own extensive test suite, not re-proven redundantly here) plus the traceability-save test. Execution Integration items 1–3 covered by §21's save-then-validate test. UI items 1–6 covered by the 9 frontend tests. Regression — see §29/§30.

29. Frontend Test Results

`npx vitest run` (full suite) -> **390 passed, 0 failed**, 21 test files (up from 385/20 before this phase: –4 for the removed `customTestingCase` block, +9 for the new `CustomTestingPage.test.tsx`). Includes every pre-existing Phase 1/Phase 2/Phase 2.1 test, unchanged and still passing.

30. Backend Test Results

`cargo test --lib` (full suite) -> **237 passed, 0 failed, 2 ignored** (pre-existing ignores, unrelated to this phase; up from 230 before this phase's `api_tests.rs` additions — 8 `custom_testing.rs` unit tests were already counted in that 230 baseline from an earlier check in this same session, +7 new `api_tests.rs` integration tests brings the final total to 237).

31. Typecheck

`npx tsc --noEmit -p .` -> **clean, zero errors.**

32. Frontend Production Build

`npx vite build` -> **succeeded** (`✓ built in 38.01s`; the same pre-existing chunk-size warning as every prior phase's report, unrelated to this phase's changes).

33. Backend Build/Check

`cargo check --lib --tests` -> **succeeded, zero errors, zero warnings** (one `unused_mut` warning was hit and fixed during this phase — see the "Errors and fixes" note below).

---

34. Live Runtime Verification

Performed against the actually-running orchestrator (`http://127.0.0.1:8080`, rebuilt from this phase's exact code via `cargo run -p aiqa-orchestrator`, reusing the pre-existing persisted SQLite store and its already-known admin key) **and** the actually-running frontend dev server (`http://localhost:3000`), driven through a real browser session — not just curl.

Case A — Complete, real requirement (curl)

```
POST /api/v1/projects/674b41e9-8195-4352-bd44-8a77057242ff/custom-testing/generate
{"requirement_text":"Check login behavior after five invalid attempts.","scope":"Authentication","skills":[{"id":

-> 200 OK
{"generation_source":"UNAVAILABLE","analysis":null,"test_cases":[],"provider":null,"model":null,
 "failure_reason":"NoCompatibleModel { task: RequirementAnalysis }"}
```

This is exactly the required disclosure. `NoCompatibleModel { task: RequirementAnalysis }` proves the real central router genuinely attempted real, task-specific routing for the newly-wired `RequirementAnalysis` task (not a generic/fake error) — this dev sandbox has 0 healthy AI providers (confirmed at backend startup: `AI infrastructure discovery complete: 4 provider(s) checked, 0 healthy`), the same pre-existing, disclosed environment limitation every prior phase's report has already documented. No strategy, no cases, no fabricated output.

Case B — Empty requirement (curl)

```
POST .../custom-testing/generate  {"requirement_text":"   ","skills":[]}
-> 400 {"error":{"code":"VALIDATION_ERROR","message":"requirement_text must not be empty", ...}}
```

Case C — RBAC (curl, live-minted Viewer key)

A real Viewer-role API key was minted live (`POST /admin/api-keys`), then used to call the endpoint:

```
-> 403 {"error":{"code":"FORBIDDEN","message":"role 'viewer' cannot perform this action (requires 'operator')", .
```

Audit log corroboration (curl)

`GET /admin/audit-log` (real, persisted, independently queried) shows, matching Case A exactly:

```
{"action":"custom_testing.generate","target":null,"detail":"NoCompatibleModel { task: RequirementAnalysis }","out
```

Real browser round-trip (Case D)

The real frontend dev server was opened in a real browser session, the real, already-verified admin session was used, the real "Phase 1 Verification" project was selected (already active), the real `Module / Scope` field was filled with `"Authentication"`, the real `Custom Testing Requirement` textarea was filled with `"Check login behavior after five invalid attempts."`, and the real `Analyze & Generate` button was clicked.

- **Real network request observed:** `POST http://localhost:3000/api/v1/projects/674b41e9-8195-4352-bd44-8a77057242ff/custom-testing/generate -> 200 OK`.
- **Real rendered page text after the response:** status badge changed from `Ready` to `AI Unavailable`, and the real reason `NoCompatibleModel { task: RequirementAnalysis }` was rendered directly on the page — matching the curl-based Case A byte-for-byte.

This proves the complete real request/response round-trip through the actual browser UI, not merely a mocked unit test — the one thing curl alone cannot prove (that the real page correctly renders the real backend's real response).

What was NOT live verified

NOT LIVE VERIFIED FOR REAL AI. A successful `REAL_AI` round-trip (real strategy + real cases from an actually-reachable model) could not be demonstrated in this sandbox — there is no local Ollama instance reachable and no cloud provider API key configured, identical to the disclosed limitation in the Phase 1 and Phase 2.1 reports for their

own AI-dependent paths. This path is instead proven correct by the dedicated unit/integration tests in §28, which use scripted fake providers precisely because no real one is reachable here — never fabricated as if it were a live call.

---

35. Fake/Stub Audit

Every changed/new file was grepped for `fake|mock|demo|sample|placeholder|stub|dummy|simulated|Math\.random|random|setTimeout|hardcoded success|hardcoded result|TODO|FIXME` (case-insensitive). Every hit reviewed:

- **Backend:** all hits are `#[cfg(test)]`-only doubles (`FakeExecutor`/`FakeDiscovery`/provider id `"mock"`) reused from the exact same pattern Phase 1/2/2.1 already established, the pre-existing, unchanged `SimulatedExecutor` (untouched by this phase), the pre-existing `sample_test_case_body` test-fixture helper (predates this phase, used by dozens of unrelated tests), or doc-comments explicitly describing what is *not* fake (e.g. "no mock, no simulated persistence layer").
- **Frontend:** all hits are the HTML `placeholder=` attribute on real form inputs (standard UX affordance showing example text before typing — exactly what the brief's own §3.D calls "Examples only"), or doc-comment references to a *different*, already-fixed prior issue elsewhere in the same file (unrelated to this phase's own edits).

Specific claims proven, per the brief's own §15 checklist:

1. **Custom requirement materially affects generation input:** yes — `augmented_requirement_text` folds the real requirement + real scope into the exact text sent to both real AI calls (`scope_is_genuinely_folded_into_the_augmented_requirement_text`, §28).
2. **No fixed generic strategy presented as custom analysis:** yes — `CustomTestingAnalysis` is only ever populated from a real, schema-validated AI response; every field is `None`/absent on any failure path (§15/§19).
3. **No static unrelated test cases claimed to be AI-generated:** yes — the case-generation half is the literal, unmodified Phase 1 function; nothing new was written that could diverge.
4. **No fake AI progress exists:** confirmed by code review — the `ANALYZING` status is set exactly once, synchronously with the real `await aiqaApi.customTesting.generate(...)` call, and cleared exactly once when that real call settles; there is no `setTimeout`, no staged/animated sequence of fake status strings anywhere in `CustomTestingPage.tsx`.
5. **AI unavailable does not produce fake AI output:** proven live, Case A/D (§34) and unit-tested (§28).
6. **Custom test execution reuses the real Phase 2.1 pipeline:** proven by §21's save-then-validate test — zero new execution code exists.
7. **No fake project/module data was introduced:** confirmed — `selectedProject?.name` renders the real project name; scope is a real, empty-by-default, honestly-labeled text input, never a fabricated module list.
8. **No fake persistence claim was introduced:** confirmed — `custom_testing.rs`/the new endpoint persist nothing; §22 documents exactly what is ephemeral vs. persisted (via the pre-existing endpoint).

---

36. Known Limitations

1. A successful `REAL_AI` round-trip could not be live-verified against a reachable model in this dev sandbox (§34) — a pre-existing, disclosed environment limitation, not new to this phase.

2. No artificial requirement-length cap was invented (§12) — if a genuinely-required limit exists at the infrastructure level (e.g. a model's context window), it is not separately enforced here beyond whatever `AiInfrastructure`/the underlying provider already does.

3. The `CustomTestingAnalysis` strategy is not itself persisted (§22) — ephemeral by design; a user who wants to keep it must copy it or rely on the `traceability.analysis_objective` field carried into a saved test case.

4. Batch custom-testing requests (multiple requirements at once) were not implemented — no existing natural batch path was found to wire, and the brief explicitly scoped batch execution to "unless it already naturally exists," which it does not here.

5. `recommended_skills` from the AI are matched to existing skills by exact case-insensitive name — a near-miss name (e.g. a synonym) is treated as a new custom skill suggestion rather than fuzzy-matched to an existing one; this is a deliberate, honest choice (no invented fuzzy-matching heuristic) rather than an oversight.

37. Remaining Work

DONE: `CustomTestingAnalysis` schema + real `RequirementAnalysis` wiring; real, unmodified test-case-generation reuse; requirement/scope/skills contract; client- and server-side validation; RBAC/project isolation; generation-source truthfulness (`REAL_AI`/`UNAVAILABLE`); real frontend UI (project context, scope, requirement, skills reuse, real status states, strategy/case rendering); real handoff into the existing Test Case Generator/execution pipeline; 8 backend unit tests + 7 backend integration tests + 9 frontend tests, full regression green (237 backend / 390 frontend); live verification (curl Cases A–C + audit log + a real browser round-trip).

PARTIAL: none identified as partial in a way not already captured by §36.

NOT IMPLEMENTED: batch custom-testing requests (§36 item 4); persistence of the strategy object itself (§36 item 3, by design).

BLOCKED: none — every in-scope item implementable within the existing architecture was implemented; the one live-verification gap (§34) is an environment-reachability limit, not an architectural blocker.

---

38. Evidence Matrix

Capability	Code	Tests	Live Verified	Verdict
Project Context	`useProject()` reused	reused, covered by existing Phase 1 tests	Case D (real project shown)	**VERIFIED**
Scope Selection	Real, honestly-labeled user-provided input	reused pattern, new UI test	Case D (real scope sent)	**VERIFIED**
Requirement Validation	Client + `api.rs` server-side	2 unit-level + 2 integration + 2 UI tests	Case A/B/D	**VERIFIED**
Skills Integration	`useSkillSelection`/`SkillsAndMcpPanel` reused; recommended-skill UX new	1 dedicated UI test	Not separately live-verified beyond code reuse (low risk — identical to Phase 1's own live-verified skill UI)	**VERIFIED**
Existing Model Router Reuse	Zero `ai_router.rs` changes; `AiTask::RequirementAnalysis` newly wired	8 unit + 7 integration tests	Case A/D (`NoCompatibleModel` proves real task-specific routing)	**VERIFIED**
Real AI Analysis	`analyze_requirement`	`a_full_real_round_trip...` (fake-provider)	**NOT LIVE VERIFIED FOR REAL AI** (§34)	**PARTIAL**
AI Output Validation	`serde_json` strict schema	`malformed_analysis_json_is_a_real_failure...`	Not separately live-verified (no real AI response reachable to malform)	**VERIFIED** (code + tests)
Strategy Generation	`CustomTestingAnalysis` rendering	1 UI test	Not live-verified with real content (§34)	**PARTIAL**
Custom Test Generation	`ai_generation::generate_test_cases` reused unmodified	inherits Phase 1's own extensive suite + 1 new UI test	Not live-verified with real content (§34)	**PARTIAL**
Test Case Pipeline Integration	`POST .../test-cases` reused	`phase3_custom_testing_generated_draft_can_be_saved_via_existing_test_case_persistence_with_real_traceability`	Confirmed by that same test's real save+validate round trip	**VERIFIED**
Execution Compiler Reuse	No new call site; automatic via saved `TestCaseRow`	covered transitively by §21's test + Phase 2.1's own extensive live verification	Not re-verified live this phase (no new code to verify)	**VERIFIED** (by construction)
Executor Reuse	No new executor	N/A (none created)	N/A	**VERIFIED** (by construction)
Evidence	Not produced by this module	N/A	N/A	**NOT APPLICABLE**
Persistence	`requirement_reference`/`traceability` round-trip	`phase3_custom_testing_generated_draft_can_be_saved_via_existing_test_case_persistence_with_real_traceability`	Same test, real HTTP round trip	**VERIFIED**
Auth/RBAC	`require_role`/`require_scoped_project` reused	2 integration tests	Live Case C (real minted Viewer key, real 403)	**VERIFIED**
No Fake Data	Fresh audit, §35	N/A	N/A	**VERIFIED**

39. Final Verdict

CONDITIONAL GO

Rationale. The complete required architecture is implemented and reused correctly: a real user requirement, with real scope and real skills, reaches a real, authorized, project-scoped backend endpoint; the real central AI router genuinely attempts real, task-specific routing for a task variant (`RequirementAnalysis`) that existed in the taxonomy but had never been wired to anything before this phase; the test-case-generation half is not a second implementation but the literal, unmodified Phase 1 function; a kept draft integrates with the existing persistence/review/execution pipeline with zero duplicated code, proven by a real save-then-validate round trip; RBAC and project isolation are enforced identically to every other AI-backed endpoint; and every honest-failure/validation/RBAC path was demonstrated live against the actually-running backend, including through a real browser session proving the real UI renders the real backend's real response correctly.

This is not scored a plain **GO** because the brief's own live-verification requirement — a real, successful `REAL_AI` round-trip producing a real strategy and real test cases from an actually-reachable model — could not be demonstrated in this sandbox (no reachable Ollama, no cloud API key configured). This is an environment-reachability limitation, not an architectural or implementation gap (the exact same limitation every prior phase's report has already disclosed for its own AI-dependent paths), and is honestly reported as **NOT LIVE VERIFIED FOR REAL AI** rather than worked around or hidden. Per the brief's own definition — *"CONDITIONAL GO if: Core real generated-case execution works, but some executor/compiler capabilities remain blocked or unavailable"* (adapted here to "core real requirement-to-pipeline flow works, but a live successful AI round-trip remains environment-blocked") — this is the accurate, non-inflated verdict.

---

40. Final Questions

1. Does the actual user requirement reach the generation backend? Yes — proven live: Case A/D's exact requirement text (`"Check login behavior after five invalid attempts."`) and scope (`"Authentication"`) were sent in the real request body and are what the real backend attempted to process (§34).

2. Does it use the existing Model Router? Yes — `custom_testing.rs` calls `AiInfrastructure::generate` directly, the same struct/method every other AI-backed endpoint (`generate-ai`, Phase 2.1's compiler) already calls. Zero changes were made to `ai_router.rs` itself (§9/§14).

3. Is a duplicate AI architecture introduced? No — no new router, no new provider registry, no new provider-discovery mechanism. `AiTask::RequirementAnalysis` is a pre-existing (if previously unused) taxonomy entry, not a newly-invented one.

4. Do selected skills materially reach generation? Yes — the caller's selected skills are converted to `ai_generation::SkillInput` and passed to both the analysis prompt (`build_analysis_prompt`) and the reused `ai_generation::generate_test_cases`, proven by `analysis_prompt_carries_the_real_requirement_scope_and_skills` (§28).

5. Are recommended skills genuinely produced or fabricated? Genuinely produced when a real AI response exists — `recommended_skills` is a real field on the schema-validated `CustomTestingAnalysis`, never computed or guessed client-side; when no real analysis exists, the UI renders no recommended-skills section at all (§17/§35).

6. How is AI output validated? Strict `serde_json` deserialization against `CustomTestingAnalysis`'s fixed shape — a missing required field is a real parse failure (§15), not silently repaired or ignored.

7. What happens when AI is unavailable? `CustomTestingOutcome::Failed{reason, analysis: None}` -> `generation_source: "UNAVAILABLE"`, `analysis: null`, `test_cases: []`, a real, specific `failure_reason` — proven live in Cases A and D (§34), byte-for-byte identical between curl and the real browser round trip.

8. Can local fallback be mistaken for AI output? No — there is no local fallback in this codebase for this capability at all (§19/§36); `generation_source` only ever takes the two real values `"REAL_AI"`/`"UNAVAILABLE"`.

9. Are generated test cases traceable to the custom requirement? Yes, in two ways: (a) every `GeneratedTestCaseDraft`'s own `source_reference`/`evidence` fields are produced by the reused, unmodified traceability-enforcement logic (`ai_generation::enforce_traceability`) against the real, augmented requirement text; (b) once saved, `requirement_reference` and `traceability` (both real, existing `TestCaseRow` fields) carry the real requirement text, scope, and analysis objective — proven by `phase3_custom_testing_generated_draft_can_be_saved_via_existing_test_case_persistence_with_real_traceability` (§28).

10. Are fixed generic test cases ever presented as custom generated output? No — every rendered case comes directly from the real (or, in tests, realistically-scripted) `test_cases` array in the actual HTTP response; there is no local template array anywhere in `CustomTestingPage.tsx`.

11. Does the Custom Testing page reuse the existing test case pipeline? Yes — via a real state handoff (`AIQAContext.activeGenerationSession`) into the completely unmodified `TestCaseGeneratorPage.tsx`, proven live at the UI-test level (§29) with a real navigation and a real render of the handed-off cases on the existing Generated Cases branch.

12. Does it reuse the existing Phase 2.1 execution pipeline? Yes, by construction — a saved Custom-Testing case is an ordinary `TestCaseRow`; no new compiler, executor, or dispatch code was written this phase (§9/§21).

13. Can a non-executable custom test case be falsely shown as executed? No — this page never claims execution at all; "Continue to Test Case Generator" only ever says exactly that, and the existing Test Case Generator's own compile-preview UI (Phase 2.1, unmodified) is what truthfully shows `READY`/`NOT_EXECUTABLE`/etc. before any real execution is attempted.

14. Is project/module data real or user-provided? Project data is real (`useProject()`); module/scope data is honestly user-provided free text, explicitly labeled as such in the UI, because the `Project` resource genuinely has no module metadata (§13).

15. Is any persistence being falsely claimed? No — §22 explicitly documents that the strategy is ephemeral and only the eventually-saved test case (via the pre-existing endpoint) is real, persisted data.

16. Are auth/RBAC boundaries preserved? Yes — identical `Role::Operator` + `require_scoped_project` checks as every other AI-backed endpoint, unit-tested and live-verified with a real, minted Viewer key (§23/§34).

17. What was live verified? A complete requirement reaching the real backend; real, task-specific router routing (`NoCompatibleModel { task: RequirementAnalysis }`); real validation rejection (empty requirement, 400); real RBAC enforcement (403 for Viewer); real audit-log corroboration; and a real browser round-trip proving the actual UI renders the actual backend's actual response correctly (§34).

18. What was NOT live verified? A successful `REAL_AI` round-trip (real strategy + real cases from an actually-reachable model) — no AI provider is reachable in this dev sandbox (§34/§36), the identical, pre-existing limitation already disclosed in Phase 1's and Phase 2.1's own reports.

19. Were any fake/stub behaviors introduced? No — the fresh audit (§35) found only pre-existing, disclosed test doubles, HTML placeholder attributes, and unrelated doc-comment references. No new fake/stub behavior exists in any Phase 3 file.

20. What remains for the next phase? Live verification of a successful `REAL_AI` round-trip once a real AI provider is reachable in some environment (no code change anticipated — this is purely an environment/credentials gap); optionally, batch custom-testing requests if a real product need emerges (§36 item 4).